

Centro Universitário da Fundação Hermínio Ometto - FHO
Engenharia da Computação - 6º semestre
Projeto Integrador

David Alves Molizane Neto - RA 116195
Eric Aparecido Savassi - RA 116822
Luis Felipe Travagim Braz Pinto - RA 116508
Joseph Costa Anjos - RA 115767
Pedro Antônio Rodrigues - RA 117776

Tarefa 3 - Mapa Funcional e Fluxo de Dados

DataPlanner: das informações em documentos para um grafo de relações da empresa

Araras, SP
Setembro de 2026

1 O problema que o DataPlanner ajuda a resolver

A informação sobre como o trabalho acontece na empresa está espalhada em documentos soltos, atas de reunião, organogramas, procedimentos e planos de projeto, quase todos em Word ou PDF e boa parte deles apenas digitalizada. O que está escrito ali nunca é cruzado: quem responde por qual projeto, qual setor depende de qual e qual documento sustenta cada processo são coisas que só existem na cabeça das pessoas ou dentro desses arquivos.

A decisão que o sistema precisa apoiar é onde estão os gargalos de relação, por exemplo um setor que concentra a aprovação de vários projetos ao mesmo tempo, ou uma única pessoa que é o único elo entre dois setores. O DataPlanner lê os documentos, transforma o que está escrito em um grafo de pessoas, setores, projetos e documentos e mostra esses pontos de concentração. Ele não decide nada sozinho, ele aponta e ordena os pontos críticos para quem vai agir.

2 Como dividimos as funções do sistema

Separamos o DataPlanner em cinco módulos, cada um com uma responsabilidade única. O grafo guardado no Neo4j não é um módulo de decisão; ele existe como apoio, é o armazenamento que os outros módulos alimentam e consultam.

Quadro 1 - Módulos do DataPlanner

Módulo	Entrada	Ação	Saída
Ingestão	Documentos da empresa em Word, PDF ou PDF digitalizado.	Extrair o texto do arquivo, usando OCR quando a página é imagem.	Texto puro, com a identificação do documento de origem.
Estruturação	Texto puro e a ontologia definida pelo grupo.	Pedir ao LLM que identifique pessoas, setores, projetos, documentos e as relações entre eles.	JSON com a lista de entidades e de relações.
Carga	JSON validado.	Percorrer o JSON e gravar cada entidade como nó e cada relação como ligação no Neo4j.	Grafo atualizado no banco.
Consulta e análise	Grafo armazenado e o recorte pedido pelo usuário (setor, projeto ou pessoa).	Buscar a vizinhança pedida, contar as ligações e ordenar os pontos que concentram mais dependências.	Lista de gargalos candidatos, com seus indicadores.
Apresentação	Lista de gargalos e indicadores.	Montar a visão do grafo e o resumo dos pontos críticos.	Tela com o grafo, a lista ordenada e os indicadores.

Fonte: elaboração do grupo.

Vale registrar uma distinção que usamos para pensar no sistema: pessoa, setor, projeto e documento são conceitos específicos do nosso cenário, mas extrair, estruturar, carregar, consultar e apresentar são funções genéricas, se um dia o DataPlanner precisasse mapear clientes e contratos em vez de setores e projetos, esses cinco módulos continuariam fazendo sentido, só mudaria o que circula dentro deles.

3 Fluxo das informações

O diagrama abaixo mostra o caminho que um dado percorre dentro do sistema. Cada seta representa o que passa de um módulo para o outro.

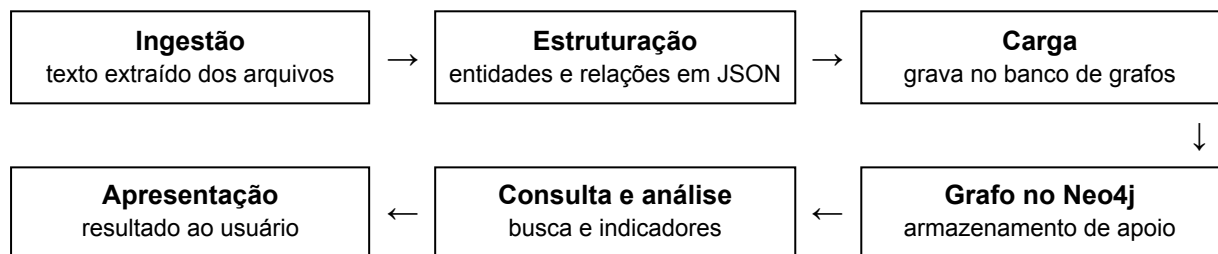


Figura 1 - Fluxo funcional do DataPlanner. Fonte: elaboração do grupo.

3.1 Acompanhando um caso do começo ao fim

Um exemplo ajuda a enxergar o fluxo funcionando na prática. Uma ata de reunião do projeto "Implantação do ERP", digitalizada em PDF, entra na ingestão e sai como texto puro, junto da referência do arquivo de origem. A estruturação envia esse texto ao LLM com a ontologia e recebe de volta um JSON em que Ana Souza aparece como Pessoa, Qualidade como Setor e Implantação do ERP como Projeto, além das relações entre eles. A carga percorre esse JSON e grava os nós e as ligações no grafo. Quando o operador consulta aquele projeto, a análise varre a vizinhança e mostra que a Qualidade é o único setor que liga a área técnica ao financeiro dentro dele, ou seja, um ponto de concentração. A apresentação destaca esse ponto na tela, e a decisão de mudar ou não o fluxo continua sendo de quem opera.

Nesse caminho, também separamos o que fica guardado do que só passa. Ficam armazenados o texto extraído de cada arquivo com a referência do documento de origem, o JSON produzido para esse documento e o grafo no Neo4j. Já a lista filtrada de gargalos e os indicadores que a acompanham são calculados na hora do pedido, servem para responder àquela consulta e não são persistidos.

4 Ferramentas que pretendemos usar e as dificuldades que já enxergamos

Esta parte vai além do mapa funcional: é o nosso planejamento de como pretendemos construir o que descrevemos acima, e de onde já conseguimos prever atrito. As disciplinas de Banco de Dados e Estruturas de Dados, que estamos cursando agora, guiaram boa parte dessas escolhas.

A ideia é guardar os dados em um banco de grafos, e pensamos no Neo4j, porque o objeto de análise aqui são as relações e não os registros isolados. Uma pergunta como "quem liga o setor A ao setor B" vira um caminho a percorrer no grafo, enquanto em um banco relacional exigiria várias junções encadeadas e uma consulta difícil de escrever e de manter. A leitura dos arquivos seria feita em Python, com Tesseract fazendo o OCR das páginas que chegam como imagem, a estruturação usaria uma chamada de LLM por API, o JSON seria o formato intermediário entre a estruturação e a carga, e a carga usaria o driver oficial do Neo4j.

A ontologia inicial que combinamos é pequena de propósito, para caber em uma chamada de LLM e ser fácil de validar. São quatro tipos de nó, Pessoa, Setor, Projeto e Documento, e quatro tipos de relação: PERTENCE_A, entre uma pessoa e o seu setor; RESPONSAVEL_POR, entre uma pessoa e o projeto que ela conduz; PARTICIPA_DE, entre um setor e um projeto; e CITA, entre um documento e a entidade que ele menciona.

4.1 O ponto que mais nos preocupa: manter a saída do LLM dentro da ontologia

O LLM é a peça mais frágil do fluxo porque a resposta dele não é determinística. O mesmo texto pode voltar com um campo a mais, com um tipo de relação que não existe na nossa ontologia ou com um JSON que sequer abre, e nada disso aparece como erro antes da hora de gravar. Por isso a saída de cada chamada precisa ser validada contra a ontologia antes de virar carga, e o que não passar na validação vai para revisão manual em vez de entrar no grafo. Tem também a questão do tamanho: documento grande não cabe em uma chamada só, então precisamos dividir o texto em trechos, e aí a mesma entidade aparece em pedaços diferentes, cada um com a sua própria resposta.

O problema irmão é a mesma entidade escrita de formas diferentes, "Ana Souza" e "A. Souza", "Setor de Qualidade" e "Qualidade", que sem tratamento vira nó duplicado e estraga a contagem de ligações, justamente o número que usamos para apontar gargalo. A ideia é normalizar o nome e usar uma chave natural na hora da carga, com MERGE em vez de CREATE, para que reprocessar o mesmo

documento não duplique nada. Soma-se a isso o OCR, que erra em nome próprio e embaralha a leitura de tabela e de organograma, empurrando erro para dentro do LLM antes mesmo de a estruturação começar.

Depois que a carga termina, a ideia para a consulta e análise é ordenar os gargalos pelo número de ligações e de dependências que cada nó concentra, para que os pontos mais críticos apareçam primeiro na lista. Para a apresentação, pensamos em uma interface web simples em HTML/CSS, sem preocupação estética, só clareza para quem vai olhar o grafo e decidir. Para organizar o trabalho entre as cinco pessoas do grupo, pretendemos usar Git/GitHub para o código e um quadro Kanban (Trello ou GitHub Projects) para dividir tarefas.

Nada disso está implementado ainda, e as dificuldades que descrevemos com o OCR, com a saída do LLM e com a carga no grafo são previsão nossa, não algo que já testamos na prática, é o plano que vamos seguir e ajustar nas próximas etapas, conforme formos avançando em Banco de Dados e Estruturas de Dados.

5 Revisando a organização que propusemos

Usamos quatro perguntas para conferir se a divisão em módulos faz sentido, e essa revisão nos deixou confortáveis com o resultado:

- **Coesão:** cada módulo trata de um assunto só, a ingestão não interpreta o conteúdo que extrai, a estruturação não grava no banco, a carga não analisa nada.
- **Acoplamento:** baixo. Cada módulo recebe só o formato de que precisa, arquivo, texto, JSON ou grafo, o que nos permite trocar o OCR ou o modelo de LLM sem mexer no resto.
- **Integração:** está clara. O Quadro 1 mostra exatamente o que cada seta do fluxograma carrega de um módulo para o próximo.
- **Caráter genérico:** extrair, estruturar, carregar, consultar e apresentar sobrevivem a uma troca de domínio; só as entidades específicas (pessoa, setor, projeto) mudariam.

Com isso, também conferimos os pontos exigidos nesta etapa: cada função aparece em um único módulo; está claro o que fica armazenado, o texto extraído, o JSON e o grafo, e o que passa de módulo para módulo; é possível seguir um documento desde o arquivo até o gargalo apontado na tela; e busca, ordenação e priorização estão identificadas como necessidades futuras, sem implementação nesta etapa.

6 Considerações finais

Este documento fecha a etapa conceitual do DataPlanner: definimos os cinco módulos, o caminho que a informação percorre do documento até o grafo e um caso completo, de uma ata digitalizada ao gargalo apontado. A partir daqui, o próximo passo é fechar a ontologia com o que estamos vendo em Banco de Dados e implementar, com Estruturas de Dados, a busca e a ordenação sobre o grafo, que hoje existem só no papel. No horizonte do projeto, essa base de conhecimento em grafo fica disponível como fonte para agentes de IA especialistas da empresa consultarem quando precisarem saber como a empresa funciona.